TP2 – Segmentation d'organe avec un U-Net 3D

Apprentissage profond en imagerie médicale – Master de mathématiques

Informations générales

— **Durée** : 4 heures (dont environ 1h30 de temps de calcul à lancer en arrière-plan), en binôme.

— **Prérequis** : TP1 (lecture, orientation, espacement, séparation par patient) ; CM2 (U-Net, pertes, métriques).

— **Environnement** : Python, `PyTorch`, `MONAI`, `numpy`, `scipy`, `matplotlib`, `pandas`. Un GPU est recommandé (Google Colab convient) ; une version réduite pour CPU est indiquée en fin d'énoncé.

— **Livrable** : un notebook commenté (code, courbes, tableaux, réponses aux questions marquées $\star$) et le fichier de séparation train/validation/test.

Objectifs. Construire de bout en bout un pipeline de segmentation volumique, comparer trois fonctions de coût (entropie croisée, Dice, Dice + entropie croisée) à protocole identique, évaluer correctement les résultats (recouvrement, distance de contour en mm, volume), puis analyser la *confiance* du modèle (calibration, seuil optimal) et ses échecs.

Données. Une tâche du *Medical Segmentation Decathlon* (par exemple `Task09_Spleen`, 41 volumes CT avec masque de rate), la même qu'au TP1. On reprend **exactement la séparation par patient** du TP1 (fichier `split_patients.csv`, 60/20/20) : le jeu de test n'est utilisé qu'à la fin, *une seule fois*.

1 Pipeline de données

A1. Construire les listes de dictionnaires `{"image": ..., "label": ...}` pour chaque ensemble à partir de `split_patients.csv`. Vérifier qu'aucun patient n'apparaît dans deux ensembles.

A2. **[À faire]** Écrire deux chaînes de transformations MONAI (dictionnaires) :

— *communes* : chargement, canal en premier, orientation RAS, rééchantillonnage à (1,5, 1,5, 2,0) mm (trilinéaire pour l'image, plus proche voisin pour le masque), fenêtrage de $[-57, 164]$ HU vers $[0, 1]$, découpage autour du premier plan (`CropForegroundd`) ;

— *entraînement* (en plus) : extraction aléatoire de patchs 96^3 avec échantillonnage préférentiel du premier plan (`RandCropByPosNegLabeld`), transformation affine légère (rotations de $\pm 15°$, échelle $\pm 10\,\%$), bruit gaussien, variation d'intensité.

Justifier chaque choix de paramètre en une phrase (fenêtre, espacement, taille de patch).

A3. Augmentations plausibles. Parmi les augmentations suivantes, lesquelles est-il raisonnable d'utiliser pour la rate : miroir gauche/droite, miroir haut/bas, rotation de 90°, rotation de $\pm 15°$, bruit gaussien, flou, variation de contraste, déformation élastique forte ? Justifier.

A4$\star$. Échantillonnage des patchs. Mesurer empiriquement, sur 100 patchs tirés dans un volume d'apprentissage, la proportion de voxels de premier plan dans un patch (i) tiré *uniformément* et (ii) tiré avec `pos=1, neg=1`. Que se passerait-il, au niveau de la perte, si l'on n'utilisait que (i) ? Que change le choix de `pos/neg` sur la probabilité prédite de premier plan ?

```python
from monai.transforms import (Compose, LoadImaged, EnsureChannelFirstd, Orientationd,
    Spacingd, ScaleIntensityRanged, CropForegroundd, RandCropByPosNegLabeld,
    RandAffined, RandGaussianNoised, RandScaleIntensityd, EnsureTyped)

common = [
    # TODO: LoadImaged, EnsureChannelFirstd, Orientationd(axcodes="RAS"),
    #       Spacingd(pixdim=(1.5, 1.5, 2.0), mode=("bilinear", "nearest")), ...
]
train_tf = Compose(common + [
    # TODO: RandCropByPosNegLabeld(..., spatial_size=(96, 96, 96), pos=1, neg=1,
    num_samples=4)
    #       RandAffined, RandGaussianNoised, RandScaleIntensityd, EnsureTyped
])
val_tf = Compose(common + [EnsureTyped(keys=["image", "label"])])
```

2 Modèle

B1. Instancier un U-Net 3D (`monai.networks.nets.UNet`) : 1 canal d'entrée, 2 canaux de sortie, `channels=(16,32,64,128,256)`, `strides=(2,2,2,2)`, `num_res_units=2`, normalisation d'instance. Afficher le nombre de paramètres et vérifier la forme de sortie sur un tenseur aléatoire $(2, 1, 96, 96, 96)$.

B2⋆. Champ réceptif. Pour une architecture simplifiée (4 niveaux d'encodeur, deux convolutions 3^3 de pas 1 par niveau suivies d'un *max-pooling* 2^3 de pas 2, puis deux convolutions 3^3 au fond), calculer le champ réceptif au niveau du goulot d'étranglement à l'aide de la récurrence $r_{\text{out}} = r_{\text{in}} + (k-1)\, j_{\text{in}}$, $j_{\text{out}} = j_{\text{in}}\, s$ (r : champ réceptif, j : saut en voxels entre deux sorties voisines). Le comparer à la taille du patch (96 voxels) et à celle du volume dans chaque direction. Quelles conséquences pour l'inférence près du bord d'un patch ?

B3⋆. Avec un lot de taille 2, pourquoi préfère-t-on la normalisation d'instance (ou de groupe) à celle par lot ?

3 Fonctions de coût et entraînement

C1. Dice douce et gradient. Écrire en PyTorch la perte

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i p_i g_i}{\sum_i p_i + \sum_i g_i}$$

(cas binaire, sans lissage). Vérifier à l'aide de `autograd` que le gradient vaut

$$\frac{\partial \mathcal{L}_{\text{Dice}}}{\partial p_j} = -\frac{2\,(g_j U - I)}{U^2}, \qquad I = \sum_i p_i g_i,\ U = \sum_i p_i + \sum_i g_i,$$

puis comparer votre perte à `monai.losses.DiceLoss` (en tenant compte de son terme de lissage).

C2. Trois modèles. Entraîner le même U-Net, avec les mêmes graines, les mêmes données et le même nombre d'itérations, pour trois fonctions de coût : (i) entropie croisée, (ii) Dice, (iii) Dice + entropie croisée (`DiceCELoss`). Optimiseur AdamW (`lr` $= 2 \cdot 10^{-4}$, `weight_decay` $= 10^{-5}$), N itérations avec décroissance polynomiale ou cosinus du pas ; N selon le matériel (voir la fin de l'énoncé). Évaluer sur la validation, par fenêtre glissante, tous les 5 passages sur les données. Sauvegarder le meilleur point de contrôle de chaque modèle.

C3. Tracer, pour les trois modèles, la perte d'entraînement et le Dice de validation en fonction du nombre d'itérations. Reporter le temps de calcul.

C4⋆. Pourquoi le déséquilibre pose problème à l'entropie croisée seule ? Estimer, sur un volume entier, la proportion de voxels de premier plan et la valeur de la perte pour un prédicteur « fond partout ». Expliquer pourquoi, avec des patchs tirés uniformément, un modèle peut converger vers une solution triviale, et ce que corrigent (ou non) le sur-échantillonnage du premier plan, la pondération des classes, la perte de Dice.

```
import torch
def soft_dice_loss(p, g):
    # p: probabilities of the foreground class, g: binary labels (same shape)
    I = (p*g).sum()
    U = p.sum() + g.sum()
    return 1 - 2*I/U

# TODO: check the analytic gradient -2*(g*U - I)/U**2 against p.grad
```

4 Inférence et post-traitement

D1. Appliquer le meilleur modèle à un volume de validation complet avec `sliding_window_inference` (fenêtre 96^3, recouvrement 0,5, pondération gaussienne), puis convertir la sortie en probabilité de premier plan (softmax) et en masque (`argmax`).

D2. Agrégation à la main. Pour comprendre ce que fait la fenêtre glissante, écrire en `numpy` une fonction `sliding_window(vol, roi, predictor, overlap, gaussian)` qui découpe le volume en fenêtres recouvrantes (avec une dernière fenêtre alignée sur le bord), applique `predictor` à chacune et recombine les prédictions par moyenne pondérée $\hat{p}(v) = \sum_j w_j(v) p^{(j)}(v) / \sum_j w_j(v)$. Vérifier que, pour un « modèle » identité (`predictor = lambda patch: patch[None]`), la reconstruction est *exacte* (à la précision machine), avec et sans pondération gaussienne, y compris pour un volume plus petit que la fenêtre. Comparer ensuite, sur un volume réel, votre agrégation à celle de MONAI.

D3. Post-traitement. Ne conserver que la plus grande composante connexe du masque prédit (connectivité 26). Mesurer son effet sur le Dice et sur la distance de Hausdorff 95 pour les trois modèles.

D4⋆. Pourquoi la pondération gaussienne améliore-t-elle l'inférence ? Relier à la réponse de B2. Dans quel cas le post-traitement par plus grande composante connexe est-il *dangereux* ?

5 Évaluation

E1. Écrire (ou compléter) les fonctions suivantes sur des masques binaires 3D avec un espacement donné en mm : `dice`, `iou`, `hd_percentile` (distance de Hausdorff au 95e centile, calculée sur les *surfaces* avec une transformée en distance euclidienne tenant compte de l'espacement), `assd` (distance moyenne de surface), `nsd` (Dice de surface normalisé, tolérance $\tau = 2$ mm), `volume_ml`. Les valider sur des ellipsoïdes décalés d'une distance connue.

E2. Calculer ces métriques pour chaque patient du jeu de *validation*, pour chaque fonction de coût, avec et sans post-traitement. Présenter un tableau (moyenne, écart-type, médiane) et des diagrammes en boîte **par patient** (une distribution, pas une moyenne seule).

E3. Volumes. Tracer le volume prédit en fonction du volume de référence (droite $y = x$) et le diagramme de Bland–Altman associé. Y a-t-il un biais systématique ?

E4⋆. Dice et distances. Construire deux masques d'une même référence : un masque décalé de quelques mm, et un masque correct auquel on ajoute un petit îlot éloigné. Calculer Dice, HD (maximum) et HD95. Que montre l'exemple sur la sensibilité de chaque métrique ? Quelle métrique choisiriez-vous pour la planification de radiothérapie ? pour une mesure de volume ?

E5⋆. Pourquoi calcule-t-on les métriques dans l'espace rééchantillonné, et quelle précaution faut-il prendre pour une évaluation « officielle » ?

6 Confiance, calibration et seuil

F1. Pour chaque modèle, tracer l'histogramme (échelle logarithmique) des probabilités de premier plan prédites sur les voxels de la validation. Quel modèle est le plus « tranché » (probabilités proches de 0 ou 1) ?

F2. Diagramme de fiabilité et ECE. Calculer, par intervalles de probabilité, la fréquence observée de premier plan en fonction de la probabilité moyenne prédite, et l'erreur de calibration espérée

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{n} \left|\mathrm{acc}_b - \mathrm{conf}_b\right|.$$

Pour éviter que le fond trivial n'écrase la mesure, restreindre le calcul à une région d'intérêt : les voxels situés à moins de 5 voxels de l'union du masque prédit et du masque de référence.

F3. Seuil de décision. Pour le modèle entraîné avec l'entropie croisée, faire varier le seuil sur la probabilité de premier plan de 0,1 à 0,9 et tracer le Dice de validation en fonction du seuil. Comparer le seuil optimal à $\frac{1}{2}F^\star$ où $F^\star$ est le Dice maximal (CM2). Simuler d'abord sur des données synthétiques (probabilités calibrées tirées d'une loi Beta, étiquettes de Bernoulli) pour vérifier la règle.

F4. Mise à l'échelle de température. Pour un modèle, chercher $T > 0$ minimisant la log-vraisemblance négative sur la validation de $\mathrm{softmax}(z/T)$ (z : logits). Mesurer l'effet sur l'ECE et sur le Dice. La température change-t-elle le masque obtenu par `argmax` ?

F5⋆. Pourquoi les modèles entraînés avec une perte de Dice sont-ils typiquement sur-confiants ? Que cela implique-t-il pour la quantification d'incertitude (CM5) ?

7 Analyse des erreurs

G1. Identifier, pour le meilleur modèle, les trois patients de validation les moins bien segmentés. Afficher coupes et masques (référence et prédiction) et décrire le type d'erreur : faux positifs isolés, sous-segmentation d'une extrémité, fuite dans un organe voisin, masque vide, erreur d'annotation possible.

G2. Tracer le Dice en fonction du volume de l'organe : les petits organes sont-ils plus difficiles ?

G3⋆. Synthèse. Quelle fonction de coût retiendriez-vous pour (i) un usage de volumétrie, (ii) un usage de détection, (iii) un usage où l'on a besoin de probabilités calibrées ? Quelles sont les limites de votre évaluation (taille du jeu de validation, variabilité inter-experts, un seul centre) ? Quel serait le protocole pour estimer correctement l'erreur de généralisation sur de nouveaux patients ?

8 Évaluation finale sur le jeu de test

Une fois les choix faits (perte, post-traitement, seuil, température) sur la *validation*, évaluer **une seule fois** le modèle retenu sur le jeu de test et rapporter, avec intervalles de confiance par *bootstrap sur les patients*, Dice, HD95 et erreur de volume.

Version réduite (CPU ou séance courte)

— Utiliser un sous-ensemble de volumes (par exemple 10 en entraînement, 4 en validation), un espacement plus grossier (3,0, 3,0, 3,0) mm, des patchs 64^3, `channels=(8,16,32,64)`, `strides=(2,2,2)`.

— $N \approx$ quelques centaines d'itérations par modèle, validation toutes les 50 itérations.

— Les valeurs numériques attendues sont alors nettement inférieures à celles du protocole complet ; les comparaisons qualitatives entre pertes restent significatives.

Éléments attendus dans le compte rendu

Partie	Livrable principal
A	pipeline commenté, mesure du premier plan par patch
B	nombre de paramètres, calcul du champ réceptif
C	vérification du gradient du Dice, courbes pour trois pertes
D	agrégation par fenêtres vérifiée sur le modèle identité, effet du post-traitement
E	tableau et boîtes à moustaches par patient, Bland–Altman
F	histogrammes, ECE, seuil optimal, température
G	analyse des échecs, discussion
Test	évaluation unique avec intervalles de confiance